Transformer From Scratch - II

Muaaz Raza (*yourssharky@gmail.com*)

Abstract:

This document is the second part of a two-part series on a from-scratch implementation of the Transformer architecture in NumPy. Building on the architecture, forward pass, and design decisions established in Part 1, this document derives the backward pass for every component of the model, including the gradients for self-attention, the feedforward block, layer normalization, and the embedding and unembedding layers. It concludes with an empirical demonstration of the model trained on the Tiny Shakespeare dataset, analyzing loss convergence and qualitative text generation.

1. Introduction / Recap

Part 1 of this series established the architecture and forward pass of this implementation: a decoder-only, causally masked Transformer built entirely from native Python and NumPy, following the general structure of Vaswani et al. (2017) and Karpathy's nanoGPT. It introduced the model's notation, derived the mathematical formulation of each component — token embedding, positional encoding, self-attention, layer normalization, the feedforward block, and the final output projection — and discussed the reasoning behind several specific design decisions, including pre-LN normalization placement, causal masking, and the use of separate per-head weight matrices.

This document assumes familiarity with that material and builds directly on it. Rather than requiring the reader to cross-reference Part 1 throughout, each forward-pass equation is restated at the start of its corresponding section here, immediately before its gradient is derived — so that each section remains self-contained. Notation, hyperparameters, and shapes are otherwise carried over unchanged from Part-1.

2. Mathematical Preliminaries

2.1 The Chain Rule

Let ***f*** be a composite function of several intermediate variables, each of which is itself a function of a shared variable ***t***:

$$f = f\big(y_1(t), y_2(t), \cdots, y_n(t)\big)$$

*The derivative of **f** with respect to **t** sums the contribution through each intermediate variable:*

$$\frac{\partial f}{\partial t} = \sum_{i=1}^{n} \frac{\partial f}{\partial y_i} . \frac{\partial y_i}{\partial t}$$

This is the familiar multivariable chain rule: to find how ***f*** changes as ***t*** changes, account for every path by which t's influence reaches ***f***.

Extension to matrices

The gradients in this document operate on matrices, not scalars. Every operation used in this architecture processes each row independently — row i of an output depends only on row i of its input — so the chain rule can be applied one row at a time, then stacked back into a matrix.

For a linear operation ***Y = X·W*** , this collapses to a single matrix identity.

Define:

$$dY = \partial L/\partial Y$$

$$dX = \partial L/\partial X$$

$$dX = dY \cdot W^T \quad (2.1.1)$$

$$dW = X^T \cdot dY \quad (2.1.2)$$

These two identities recur throughout this document, since nearly every learnable parameter (Wq, Wk, Wv, Wp, W0, W1, Wu) enters the forward pass through exactly this ***X·W*** pattern. For components whose row-wise computation is nonlinear — including Layer Normalization and GELU — the same row-independence principle applies, but each row's local derivative must be recomputed individually rather than reused as a single constant matrix; this is covered in each component's own derivation.

2.2 The ADAM Optimizer

Motivation

Plain gradient descent updates each parameter using only the current gradient, with no memory of the gradient's history. This becomes problematic near a minimum, where gradient magnitudes shrink and updates correspondingly slow to a crawl — the same fixed learning rate that made reasonable progress early in training becomes inefficient once gradients are small. It also treats every parameter identically, applying the same learning rate regardless of how consistently or erratically that parameter's gradient has behaved over time. Adam addresses both issues by maintaining, for each parameter individually, a running estimate of the gradient's mean (first moment) and its variance (second moment) across past updates — using this history to adaptively scale each parameter's update, rather than relying on the current gradient and a single fixed learning rate alone.

Adam Formula:

$$m_t = \beta_1 * m_{t-1} + (1 - \beta_1) * dW$$

$$v_t = \beta_2 * v_{t-1} + (1 - \beta_2) * (\mathrm{dW} \otimes dW)$$

$$\hat{m}_t = \frac{m_t}{1-\beta_1^t}$$

$$\hat{v}_t = \frac{v_t}{1-\beta_2^t}$$

$$W_t = W_{t-1} - \frac{\alpha * \hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon}$$

This update rule is applied independently to every learnable parameter in the model — including the weight matrices (Wq, Wk, Wv, Wp, W0, W1, Wu, the embedding table) and the Layer Normalization scale and shift parameters (γ, β, for each of the three normalization instances) — replacing a direct, unscaled subtraction of the gradient.

where:

- dW = the gradient of the loss with respect to a given parameter, matching the notation established in Section 2.1 (used generically here to denote any learnable parameter, not only a weight matrix)
- $dW \otimes dW$ = the element-wise square of dW, not a matrix product
- β1, β2 = hyperparameters controlling the decay rate of the first and second moment estimates; this implementation uses β1 = 0.9, β2 = 0.999
- t = the current iteration (update step) number
- α = the learning rate; this implementation uses α = 0.001
- ε = a small constant preventing division by zero; this implementation uses ε = 1e-8

2.3 Softmax gradient

Formula

Let X be a matrix of shape (m, n), m = seq_len, n = d_model . Softmax is applied to each entry $x_{i,j}$, normalizing across the columns of each row:

$$Y = Softmax(X)$$

$$y_{i,j} = \frac{e^{x_{i,j}}}{\sum_{k=1}^{n} e^{x_{i,k}}} \quad (1)$$

Here $x_{i,j}$ denotes the entry at row i, column j of X. The sum in the denominator runs across the n columns of row i — each row is normalized independently of every other row, consistent with how softmax is used throughout this architecture (Part 1, Section 2.1).

Gradient

Let dY be a matrix of the same shape as Y, where each entry is the partial derivative of the loss with respect to the corresponding entry of Y:

$$dy_{i,j} = \frac{\partial L}{\partial y_{i,j}}$$

The goal is to find $dX = \partial L/\partial X$, given dY. The key observation, visible directly from equation (1), is that a single input entry $x_{i,j}$ does not only affect its own output $y_{i,j}$ — because the denominator sums over the entire row, $x_{i,j}$ affects every output in row i, not just $y_{i,j}$. This means finding $dx_{i,j}$ requires summing the contribution through every output in that row, via the chain rule:

$$dx_{i,j} = \frac{\partial L}{\partial x_{i,j}} = \sum_{k=1}^{n} d\,y_{i,k} \cdot \frac{\partial y_{i,k}}{\partial x_{i,j}} \qquad (2)$$

To evaluate this sum, $\partial y_{i,k}/\partial x_{i,j}$must be found for every k — and this derivative takes a different form depending on whether k equals j or not, so the two cases are handled separately below.

To keep the algebra uncluttered, let S_i denote the denominator of row i:

$$S_i = \sum_{k=1}^{n} e^{x_{i,k}}$$

Case 1: k = j — differentiating an output with respect to the same-indexed input that produced it.

$$\frac{\partial y_{i,\mathrm{j}}}{\partial x_{i,\mathrm{j}}} = \frac{\partial}{\partial x_{i,j}}\left(\frac{e^{x_{i,j}}}{\sum_{k=1}^{n} e^{x_{i,k}}}\right) = \frac{\partial}{\partial x_{i,j}}\left(\frac{e^{x_{i,j}}}{S_i}\right)$$

Applying the quotient rule:

$$\frac{\partial y_{i,\mathrm{j}}}{\partial x_{i,\mathrm{j}}} = \left(\frac{S_i * \frac{\partial}{\partial x_{i,j}} e^{x_{i,j}} - \frac{\partial S_i}{\partial x_{i,j}} * e^{x_{i,j}}}{S_i^2}\right) \qquad (3)$$

Since $e^{x_{i,j}}$ appears as one term in the sum defining S_i, differentiating both the numerator and S_i itself with respect to $x_{i,j}$ gives the same result:

$$\frac{\partial}{\partial x_{i,j}} e^{x_{i,j}} = e^{x_{i,j}} \quad , \quad \frac{\partial S_i}{\partial x_{i,j}} = e^{x_{i,j}}$$

Substituting into (3):

$$\frac{\partial y_{i,j}}{\partial x_{i,j}} = \frac{S_i \cdot e^{x_{i,j}} - e^{x_{i,j}} \cdot e^{x_{i,j}}}{S_i^2} = \frac{e^{x_{i,j}}(S_i - e^{x_{i,j}})}{S_i^2} = \frac{e^{x_{i,j}}}{S_i}\left(1 - \frac{e^{x_{i,j}}}{S_i}\right)$$

Recognizing $e^{x_{i,j}}/S_i$ as $y_{i,j}$ itself, per equation (1):

$$\frac{\partial y_{i,\mathrm{j}}}{\partial x_{i,\mathrm{j}}} = y_{i,j}\left(1 - y_{i,j}\right) \tag{4}$$

Case 2: k ≠ j — differentiating an output with respect to a *different* input, one that only appears in the shared denominator.

$$\frac{\partial y_{i,\mathrm{k}}}{\partial x_{i,\mathrm{j}}} = \frac{\partial}{\partial x_{i,j}}\left(\frac{e^{x_{i,k}}}{S_i}\right) = \left(\frac{S_i * \frac{\partial}{\partial x_{i,j}} e^{x_{i,k}} - \frac{\partial S_i}{\partial x_{i,j}} * e^{x_{i,k}}}{S_i^2}\right)$$

Here, the numerator $e^{x_{i,k}}$ does not depend on $x_{i,j}$ at all (k ≠ j), so its derivative is zero; only S_i still depends on $x_{i,j}$, contributing $e^{x_{i,j}}$

$$\frac{\partial}{\partial x_{i,j}} e^{x_{i,k}} = 0, \qquad \frac{\partial S_i}{\partial x_{i,j}} = e^{x_{i,j}}$$

Substituting:

$$\frac{\partial y_{i,k}}{\partial x_{i,j}} = \frac{S_i \cdot 0 \; - \; e^{x_{i,j}} \cdot e^{x_{i,k}}}{S_i^2} = -\frac{e^{x_{i,j}}}{S_i} \cdot \frac{e^{x_{i,k}}}{S_i}$$

$$\frac{\partial y_{i,\mathrm{j}}}{\partial x_{i,\mathrm{j}}} = -y_{i,k} . y_{i,j} \tag{5}$$

Combining both cases:

Substituting (4) and (5) back into the chain-rule sum from equation (2) — separating out the k = j term, which uses (4), from every k ≠ j term, which uses (5):

$$dx_{i,j} = \sum_{k \neq j} d\,y_{i,k} \cdot \left(-y_{i,j} \cdot y_{i,k}\right) \; + \; dy_{i,j} \cdot y_{i,j}\left(1 - y_{i,j}\right)$$

Factoring out $y_{i,j}$ from every term:

$$dx_{i,j} = -y_{i,j}\left(\sum_{k \neq j} d\,y_{i,k} \cdot y_{i,k} \; - \; dy_{i,j} + dy_{i,j} \cdot y_{i,j}\right)$$

The k = j term, $dy_{i,j} \cdot y_{i,j}$, can be folded back into the sum (since the sum was only excluding k = j to keep the two cases separate — adding it back turns the restricted sum into a sum over all k):

$$dx_{i,j} = y_{i,j}.\left(dy_{i,j} - \sum_{k=1}^{n} d\,y_{i,k}.\left(y_{i,k}\right)\right)$$

This is the gradient of the loss with respect to a single input entry $x_{i,j}$, expressed entirely in terms of the forward-pass output Y and the upstream gradient dY — no reference to the raw input X or its exponentials remains, which is typical of softmax's backward pass and makes it cheap to compute.

Extension to full matrix:

Equation (6) contains a term, $\Sigma_k dy_{i,k} \cdot y_{i,k}$, that is the same for every j within a given row i — it depends only on row i as a whole, not on which column j is being differentiated. This term is precisely the row-wise dot product of dY and Y: element-wise multiply the two matrices, then sum along each row. Define this as a column vector Sd, of shape (m, 1), holding one such value per row:

$$\mathrm{Sd}_{i,1} = \sum_{j=1}^{n} d\,y_{i,j}.y_{i,j}$$

Since Sd_i is constant across a row, equation (6) can be applied to every entry of the matrix at once, using broadcasting to subtract each row's scalar Sd_i from every entry in that row of dY:

$$d\mathrm{X} = \frac{\partial L}{\partial \mathrm{X}} = \; \mathrm{Y} * (\mathrm{dY} - \mathrm{Sd}) \qquad \text{(A)}$$

where * denotes element-wise multiplication, and $(dY - Sd)$relies on Sd broadcasting across each row. Equation (A) is the final expression for the gradient with respect to X, the input to the softmax function.

Here:

- Y = the output matrix of the softmax function (the forward-pass result)
- dY = the upstream gradient, ∂L/∂Y, matching the shape of Y
- Sd = a column vector, shape (m, 1), holding the row-wise dot product of Y and dY, broadcast against dY in equation (A)

Numpy Implementation

```
def softmaxGrad(Y, dY):
    return Y * (dY - np.sum(Y*dY, axis=-1, keepdims=True))
```

2.4 Layer-Normalization gradient

Formula:

Let X be the input matrix to Layer Normalization, of shape (m, n), m = seq_len, n = d_model , and Y = LayerNorm(X) the output matrix of the same shape. Each row is normalized independently, using its own mean and variance across the n feature columns, then rescaled by learnable parameters γ and β:

$$\hat{x}_{i,j} = \frac{x_{i,j} - \mu_i}{\sqrt{\sigma_i^2 + \varepsilon}}, y_{i,j} = \hat{x}_{i,j} * \gamma_j + \beta_j \quad (2.4.1)$$

where:

- $\mu_i = (1/n) \cdot \Sigma_j x_{i,j}$ — the mean of row i
- ${\sigma_i}^2 = (1/n) \cdot \Sigma_k (x_{i,k} - \mu_i)^2$ — the variance of row i
- γ, β = learnable scale and shift parameters, shape (1, n), shared across all rows
- ε = a small constant preventing division by zero (this implementation uses ε = 1e-7)

Gradient :

Given dY := ∂L/∂Y, the goal is to find dX, dγ, and dβ.

Parameter gradients. Since γ_j and β_j are shared across every row, their gradients sum the contribution from all m rows:

$$d\gamma_j = \sum_{i=1}^{m} \hat{x}_{i,j} \cdot dy_{i,j}, \qquad d\beta_j = \sum_{i=1}^{m} d\, y_{i,j}$$

Gradient into the normalized value. Since $y_{i,j} = \hat{x}_{i,j} \cdot \gamma_j + \beta_j$ is a simple linear function of$\hat{x}_{i,j}$:

$$d\hat{x}_{i,j} = dy_{i,j} \cdot \gamma_j$$

Gradient into the input, X :

This is the hard part: $x_{i,j}$ does not only affect $\hat{x}_{i,j}$ directly — it also affects μ_i and ${\sigma_i}^2$, which in turn affect every $\hat{x}_{i,k}$ in the same row. Finding $dx_{i,j}$ therefore requires the chain rule, summed over the whole row:

$$\frac{\partial L}{\partial x_{i,j}} = \sum_{k=1}^{n} d\,\hat{x}_{i,k} \cdot \frac{\partial \hat{x}_{i,k}}{\partial x_{i,j}}$$

Two intermediate derivatives are needed first, both following directly from the definitions ofμ_i and ${\sigma_i}^2$:

$$\frac{\partial \mu}{\partial x_{i,j}} = \frac{\partial}{\partial x} \frac{\sum_k^n x_{i,k}}{n} = \frac{1}{n} \qquad (2.4.3)$$

Differentiating ${\sigma_i}^2$ with respect to $x_{i,j}$

Starting from the definition:

$$\sigma_i^2 = \frac{1}{n} \sum_{k=1}^{n} \left(x_{i,k} - \mu_i\right)^2$$

Differentiate term by term, applying the chain rule to each squared term:

$$\frac{\partial \sigma_i^2}{\partial x_{i,j}} = \frac{1}{n} \sum_{k=1}^{n} 2\left(x_{i,k} - \mu_i\right) \cdot \frac{\partial \left(x_{i,k} - \mu_i\right)}{\partial x_{i,j}}$$

Since μ_i itself depends on $x_{i,j}$ (via $\partial \mu_i / \partial x_{i,j}$= 1/n), the derivative $\partial\left(x_{i,k} - \mu_i\right)/\partial x_{i,j}$ splits into two cases:

Case k = j:

$$\frac{\partial(x_{i,j}-\mu_i)}{\partial x_{i,j}} = 1-\frac{1}{n}$$

giving, for this term:

$$\frac{2}{n}(x_{i,j}-\mu_i)\left(1-\frac{1}{n}\right) = \frac{2}{n}\left[(x_{i,j}-\mu_i)-\frac{1}{n}(x_{i,j}-\mu_i)\right]$$

Case k ≠ j:

$$\frac{\partial(x_{i,k}-\mu_i)}{\partial x_{i,j}} = -\frac{1}{n}$$

giving, for each such term:

$$\frac{2}{n}(x_{i,k}-\mu_i)\left(-\frac{1}{n}\right) = -\frac{2}{n^2}(x_{i,k}-\mu_i)$$

Combining both cases into a single sum — the k=j term contributes its full piece, and every k≠j term contributes the second form:

$$\frac{\partial\sigma_i^2}{\partial x_{i,j}} = \frac{2}{n}\left[\sum_{k\neq j}\left(-\frac{1}{n}(x_{i,k}-\mu_i)\right)+(x_{i,j}-\mu_i)-\frac{1}{n}(x_{i,j}-\mu_i)\right]$$

The k=j term's " $-1/n(x_{i,j}-\mu_i)$"piece has exactly the same form as every k≠j term, so it can be folded back into the sum, turning the restricted sum (k≠j) into a full sum over all k:

$$\frac{\partial\sigma_i^2}{\partial x_{i,j}} = \frac{2}{n}\left[\sum_{k=1}^{n}\left(-\frac{1}{n}(x_{i,k}-\mu_i)\right)+(x_{i,j}-\mu_i)\right]$$

The remaining sum vanishes, since by definition the deviations from the mean always sum to zero:

$$\sum_{k=1}^{n}(x_{i,k}-\mu_i) = \sum_{k=1}^{n}x_{i,k}-n\mu_i = n\mu_i-n\mu_i = 0$$

leaving the final result:

$$\frac{\partial\sigma_i^2}{\partial x_{i,j}} = \frac{2}{n}(x_{i,j}-\mu_i) \qquad (2.4.4)$$

Differentiating $\hat{x}_{i,k}$ with respect to $x_{i,j}$ then splits into two cases:

Case k = j (an output's gradient with respect to the same input that produced it):

Starting from the definition:

$$\hat{x}_{i,j} = \frac{x_{i,j} - \mu_i}{\sqrt{\sigma_i^2 + \varepsilon}}$$

This is a quotient (or equivalently, a product of$\left(x_{i,j} - \mu_i\right)$ and$(\sigma_i{}^2 + \varepsilon)^{-1/2}$); applying the product rule:

$$\frac{\partial \hat{x}_{i,j}}{\partial x_{i,j}} = \underbrace{\frac{\partial\left(x_{i,j} - \mu_i\right)}{\partial x_{i,j}}}_{1-\frac{1}{n}} \cdot \frac{1}{\sqrt{\sigma_i^2 + \varepsilon}} + \left(x_{i,j} - \mu_i\right) \cdot \underbrace{\frac{\partial}{\partial x_{i,j}}\left(\sigma_i^2 + \varepsilon\right)^{-1/2}}_{-\frac{1}{2}\left(\sigma_i^2+\varepsilon\right)^{-3/2} \cdot \frac{2}{n}\left(x_{i,j}-\mu_i\right)}$$

using $\partial\mu_i/\partial x_{i,j} = 1/n$ from before, and equation (2.4.4) for $\partial\sigma_i{}^2/\partial x_{i,j}$}. Substituting both:

$$= \left(1 - \frac{1}{n}\right)\frac{1}{\sqrt{\sigma_i^2 + \varepsilon}} - \frac{1}{n}\left(x_{i,j} - \mu_i\right)^2 \cdot \frac{1}{(\sigma_i^2 + \varepsilon)^{3/2}}$$

Distributing the first term, and rewriting the second using $\hat{x}_{i,j} = \left(x_{i,j} - \mu_i\right)/\sqrt{(\sigma_i{}^2 + \varepsilon)}$, so that $\left(x_{i,j} - \mu_i\right)^2/(\sigma_i{}^2 + \varepsilon)^{3/2} = \hat{x}_{i,j} \cdot \left(x_{i,j} - \mu_i\right)/(\sigma_i{}^2 + \varepsilon)$:

$$\frac{\partial \hat{x}_{i,j}}{\partial x_{i,j}} = \frac{1}{\sqrt{\sigma_i^2 + \varepsilon}} - \frac{1}{n} \cdot \frac{1}{\sqrt{\sigma_i^2 + \varepsilon}} - \frac{\hat{x}_{i,j}}{n} \cdot \frac{x_{i,j} - \mu_i}{\sigma_i^2 + \varepsilon}$$

Case k ≠ j (an output's gradient with respect to a different row-entry, reached only through $\mu_i$and σ_i²):

Starting from the definition, now with k ≠ j so $x_{i,k}$ does not depend on $x_{i,j}$ directly — only μ_i and $\sigma_i{}^2$ carry the dependency:

$$\hat{x}_{i,k} = \frac{x_{i,k} - \mu_i}{\sqrt{\sigma_i^2 + \varepsilon}}$$

Applying the product rule the same way:

$$\frac{\partial \hat{x}_{i,k}}{\partial x_{i,j}} = \underbrace{\frac{\partial\left(x_{i,k} - \mu_i\right)}{\partial x_{i,j}}}_{-\frac{1}{n}} \cdot \frac{1}{\sqrt{\sigma_i^2 + \varepsilon}} + \left(x_{i,k} - \mu_i\right) \cdot \underbrace{\frac{\partial}{\partial x_{i,j}}\left(\sigma_i^2 + \varepsilon\right)^{-1/2}}_{-\frac{1}{2}\left(\sigma_i^2+\varepsilon\right)^{-3/2} \cdot \frac{2}{n}\left(x_{i,j}-\mu_i\right)}$$

Substituting and simplifying, using $\hat{x}_{i,k} = \left(x_{i,k} - \mu_i\right)/\sqrt{(\sigma_i{}^2 + \varepsilon)}$ to rewrite the second term:

$$\frac{\partial \hat{x}_{i,k}}{\partial x_{i,j}} = -\frac{1}{n} \cdot \frac{1}{\sqrt{\sigma_i^2 + \varepsilon}} - \frac{\hat{x}_{i,k}}{n} \cdot \frac{x_{i,j} - \mu_i}{\sigma_i^2 + \varepsilon}$$

Notice this matches the Case k=j result exactly, except for the missing leading$1/\sqrt{(\sigma_i{}^2 + \varepsilon)}$ term — that term arose only from differentiating $\left(x_{i,j} - \mu_i\right)$ with respect to itself, which doesn't occur when k≠j.

Combining both cases.

Substituting the two local derivatives into the chain-rule sum, and separating the k=j term (using the Case k=j result) from every k≠j term (using the Case k≠j result):

$$\frac{\partial L}{\partial x_{i,j}} = \sum_{k \neq j} d\,\hat{x}_{i,k} \left(-\frac{1}{n} \cdot \frac{1}{\sqrt{\sigma_i^2 + \varepsilon}} - \frac{\hat{x}_{i,k}}{n} \cdot \frac{x_{i,j} - \mu_i}{\sigma_i^2 + \varepsilon} \right)$$
$$+ \; d\hat{x}_{i,j} \left(\frac{1}{\sqrt{\sigma_i^2 + \varepsilon}} - \frac{1}{n} \cdot \frac{1}{\sqrt{\sigma_i^2 + \varepsilon}} - \frac{\hat{x}_{i,j}}{n} \cdot \frac{x_{i,j} - \mu_i}{\sigma_i^2 + \varepsilon} \right)$$

Distributing $d\hat{x}_{i,j}$ across its bracket shows two of its three terms match the k≠j sum's general term evaluated at k=j, leaving one leftover piece, $d\hat{x}_{i,j} \cdot 1/\sqrt{(\sigma_i{}^2 + \varepsilon)}$. Folding the matching terms back in extends the restricted sum to a full sum over all k:

$$\frac{\partial L}{\partial x_{i,j}} = -\frac{1}{n} \cdot \frac{1}{\sqrt{\sigma_i^2 + \varepsilon}} \sum_{k=1}^{n} d\,\hat{x}_{i,k} \; - \; \frac{1}{n} \cdot \frac{x_{i,j} - \mu_i}{\sigma_i^2 + \varepsilon} \sum_{k=1}^{n} d\,\hat{x}_{i,k} \hat{x}_{i,k} \; + \; d\hat{x}_{i,j} \cdot \frac{1}{\sqrt{\sigma_i^2 + \varepsilon}}$$

Defining two row-wise reduction terms:

$$Sc_i = \sum_{k=1}^{n} d\,\hat{x}_{i,k}, \qquad Sd_i = \sum_{k=1}^{n} d\,\hat{x}_{i,k} \cdot \hat{x}_{i,k}$$

gives the final, simplified result:

$$dx_{i,j} = -\frac{1}{n} \left(Sc_i \cdot \frac{1}{\sqrt{\sigma_i^2 + \varepsilon}} + Sd_i \cdot \frac{x_{i,j} - \mu_i}{\sigma_i^2 + \varepsilon} \right) + d\hat{x}_{i,j} \cdot \frac{1}{\sqrt{\sigma_i^2 + \varepsilon}} \tag{2.4.5}$$

Matrix Form

Let Sc and Sd be column vectors of shape (m, 1), holding the row-wise sums above for every row. Broadcasting each across its row, equation (2.4.5) becomes:

$$dX = \frac{1}{\sqrt{\sigma^2 + \varepsilon}}\left(d\hat{X} - \frac{Sc}{n} - \hat{X} * \frac{Sd}{n}\right)$$

where σ^2 and Sc/Sd (shape (m,1)) broadcast against $d\hat{X}$ and $\hat{X}$ (shape (m,n)).

Numpy Implementation:

```
def layernormGrad( dY, Xhat, gamma, variance, epsilon):

    # dY/dgamma
    dgamma = np.sum(dY * Xhat, axis=0)

    # dY/dbeta
    dbeta = np.sum(dY , axis=0)

    # gradient through gamma
    dXhat = dY * gamma

  # row-wise reduction terms, Sc/n and Sd/n (implemented via mean)

    mean_dXhat = np.mean(dXhat, axis=1, keepdims=True) #Sc/n
    mean_dXhat_Xhat = np.mean(dXhat * Xhat,axis =1,keepdims =True) # Sd/n

    std_inv = 1.0 / np.sqrt(
        variance+ epsilon
    )

    dX = std_inv * (
        dXhat
        - mean_dXhat
        - Xhat * mean_dXhat_Xhat
    )



    return dX, dgamma, dbeta
```

Section 3 : Backpropagation

3.1 Final Layer (Softmax , Cross Entropy , Unembedding and Layer Norm)

Formula:

Let Xfn be the input matrix of shape (seq_len, d_model), where T = seq_len, to the final block. The following operations are applied in order:

$$Xlf = LayerNorm(Xfn)$$

$$Xv = Xlf.Wu$$

$$Xf = softmax(Xv)$$

$$L = Loss = -\frac{1}{T}\sum_{i=1}^{T} ln\left(Xf_{yt_i}\right)$$

where:

- T = seq_len = number of input tokens to the model
- Wu = unembedding matrix, of shape (d_model, vocab_size)
- Yt = the target vector, containing the correct next-token index at each position i
- Cross-entropy loss is used, evaluated only at the target token's probability at each position

Gradient

Backpropagation proceeds from the loss backward to the start of the block.

Step 1 — combine softmax and cross-entropy before differentiating. Rather than differentiating softmax and the loss separately, substituting softmax's definition directly into the loss expression first leads to significant cancellation:

$$\mathrm{L} = -\frac{1}{T}\sum_{i=1}^{T} ln\left(\frac{e^{xv_{i,yt_i}}}{\sum_{j=1}^{\mathrm{v}} e^{xv_{i,j}}}\right) = -\frac{1}{T}\sum_{i=1}^{T}\left(ln(e^{xv_{i,yt_i}}) - ln\left(\sum_{j=1}^{v} e^{xv_{i,j}}\right)\right)$$

$$= -\frac{1}{T}\sum_{i=1}^{T}\left(xv_{i,yt_i} - ln\left(\sum_{j=1}^{v} e^{xv_{i,j}}\right)\right)$$

using the logarithm identity $ln(a/b) = ln(a) - ln(b)$,and $ln(e^x) = x$. Here v denotes vocab_size

Step 2 — differentiate with respect to a general logit entry $xv_{i,j}$. Since xv_{i,y_i} only equals $xv_{i,j}$when j = y_i, this splits into two cases.

Differentiating the first term, xv_{i,y_i},with respect to$xv_{i,j}$: this equals 1 when j = y_i(differentiating a variable with respect to itself), and 0 when $j \neq y_i$, since xv_{i,y_i} does not depend on a different entry $xv_{i,j}$.

Differentiating the second term, $ln(\Sigma_k e^{xv_{i,k}})$,with respect to $xv_{i,j}$: this term depends on$xv_{i,j}$ regardless of which j is chosen, since $xv_{i,j}$appears inside the sum for every j. By the chain rule:

$$\frac{\partial}{\partial xv_{i,j}} \ln \sum_{k=1}^{v} e^{xv_{i,k}} = \frac{e^{xv_{i,j}}}{\sum_{k=1}^{v} e^{xv_{i,k}}} = xf_{i,j}$$

Case 1 ($j = y_i$):

$$\frac{\partial \mathrm{L}}{\partial xv_{i,j}} = -\frac{1}{T}\left(1 - \frac{e^{xv_{i,\mathrm{j}}}}{\sum_{k}^{v} e^{xv_{i,k}}}\right) = \frac{1}{T}(xf_{i,j} - 1)$$

Case 2 ($j \neq y_i$):

$$\frac{\partial L}{\partial xv_{i,j}} = -\frac{1}{T}(0 - xf_{i,j}) = \frac{1}{T} \cdot xf_{i,j}$$

Matrix form. Let H be a one-hot matrix of the same shape as Xf, with:

$$h_{i,j} = \begin{cases} 1 & j = y_i \\ 0 & j \neq y_i \end{cases}$$

Both cases combine into a single matrix expression — Case 1 subtracts 1 exactly where H is 1, and Case 2 leaves $xf_{i,j}$unchanged where H is 0:

$$dXv = \frac{\partial L}{\partial Xv} = \frac{1}{T}(Xf - H) \qquad (3.1.1)$$

Step 3 — propagate backward through the unembedding projection. $Xv = Xlf \cdot Wu$ is a matrix multiplication, so the identities from Section 2.1 apply directly:

$$dWu = Xlf^T \cdot dXv, \qquad dXlf = dXv \cdot Wu^T$$

Step 4 — propagate backward through the final Layer Normalization. Using the LayerNorm gradient derived in Section 2.4, with $dXlf$ as the upstream gradient, and the final block's own learnable parameters (γ_final, β_final — distinct from every per-layer LayerNorm's parameters):

$$dXfn,\ d\gamma_{final},\ d\beta_{final} = layernormGrad\left(dXlf,\ \hat{X}_{lf},\ \gamma_{final},\ \sigma^2_{lf},\ \varepsilon\right)$$

dXfn is the gradient handed off to the preceding Feedforward block.

NumPy Implementation:

```
        Xf[np.arange(seq_len), y] -= 1
        dXv = Xf * (1/T)  # (seq_len,vocab_size)

        dWu = Xlf.T @ dXv  # (d_model,vocab_size)
        dXlf = dXv @ Wu.T  # (seq_len,d_model)


        dXfn, dgamma_final, dbeta_final = layernormGrad(dXlf, Xhat_lf,
gamma_final, variance_lf, epsilon )
```

3.2 FFN Block

Formula

Let Xat be the input matrix of shape (seq_len, d_model), where T = seq_len, to the FFN block. The following operations are applied in order:

$$Xn2 = LayerNorm(Xat)$$
$$Z0 = Xn2 \cdot W0 + B0$$
$$A0 = GELU(Z0)$$
$$A1 = A0 \cdot W1 + B1$$
$$Xfn = A1 + Xat$$

The last expression is the residual connection, and GELU is the non-linear activation function.

where:

- W0 = weight matrix, of shape (d_model, ffn_wd)
- W1 = weight matrix, of shape (ffn_wd, d_model)
- B0 = bias vector, of shape (1, ffn_wd), broadcast across all T rows

- B1 = bias vector, of shape (1, d_model), broadcast across all T rows

Gradient:

From Section 3.1, dXfn = ∂L/∂Xfn is already known. The goal is to find dXat.

Xat appears in two separate places in the forward pass: once as the input that flows through LayerNorm and the rest of the block, and once again directly, via the residual connection. Both paths contribute to dXat, and their contributions are added together.

Path 1 — through the residual connection. Since Xfn = A1 + Xat, the gradient passes straight through unchanged:

$$dXat_1 = dA1 = dXfn$$

Path 2 — backward through the block itself. Applying the X·W identities from Section 2.1 at each linear step, and the GELU derivative (below) at the non-linearity:

$$dW1 = A0^T \cdot dA1, \qquad dB1 = \sum_{i=1}^{T} d\,A1_{i,:}$$

$$dA0 = dA1 \cdot W1^T$$

$$dZ0 = dA0 * GELU'(Z0)$$

$$dW0 = Xn2^T \cdot dZ0, \qquad dB0 = \sum_{i=1}^{T} d\,Z0_{i,:}$$

$$dXn2 = dZ0 \cdot W0^T$$

dB0 and dB1 sum their respective upstream gradients over all T rows, since B0 and B1 are broadcast identically to every row in the forward pass — the backward pass of a broadcast operation is a sum over the broadcast dimension.

Finally, propagating dXn2 backward through LayerNorm (Section 2.4), using the FFN block's own γ, β:

$$dXat_2,\ d\gamma_{ffn},\ d\beta_{ffn} = \text{layernormGrad}\left(dXn2,\ \hat{X}_{n2},\ \gamma_{ffn},\ \sigma^2_{ffn},\ \varepsilon\right)$$

Combining both paths:

$$dXat = dXat_1 + dXat_2$$

GELU Gradient

$$GELU(z) = 0.5 \cdot z \cdot (1 + \tanh(u)), \qquad u = \sqrt{\frac{2}{\pi}}(z + 0.044715 \cdot z^3)$$

Writing t := 1 + tanh(u), applying the product rule:

$$GELU'(z) = 0.5 \cdot t + 0.5 \cdot z \cdot \text{sech}^2(u) \cdot \sqrt{\frac{2}{\pi}}(1 + 3 \cdot 0.044715 \cdot z^2)$$

Note that sech² is applied to u (tanh's argument), not to t — a common point of confusion, since t already contains tanh(u) within it. This derivative is applied element-wise to every entry of Z0.

NumPy Implementation:

1.Gelu gradient

```
def dgelu(x: np.ndarray) -> np.ndarray:
    """Derivative of GELU w.r.t. x (used in FFN backward pass)."""
    inner      = (2 / np.pi) ** 0.5 * (x + 0.044715 * x ** 3)
    tanh_inner = np.tanh(inner)
    sech2      = 1 / np.cosh(inner) ** 2
    d_inner    = (2 / np.pi) ** 0.5 * (1 + 3 * 0.044715 * x ** 2)
    # chain rule: d/dx [0.5*x*(1+tanh(inner))]
    return 0.5 * (1 + tanh_inner) + 0.5 * x * sech2 * d_inner
```

2. FFN block gradients

```
dXat1 = dA1 = dXfn

dW1 = A0.T @ dA1
dB1 = np.sum(dA1, axis=0, keepdims=True)

dA0 = dA1 @ W1.T
dZ0 = dA0 * dgelu(Z0)

dW0 = Xn2.T @ dZ0
dB0 = np.sum(dZ0, axis=0, keepdims=True)
dXn2 = dZ0 @ W0.T

dXat2, dgamma_ffn, dbeta_ffn = layernormGrad(
    dXn2, Xhat_n2, gamma_ffn, variance_ffn, epsilon
```

```
)

dXat = dXat1 + dXat2
```

3.3 Attention Block:

Formula

Let Xin be the input to the attention block — the initial embedding matrix Xp, for the first layer, or the preceding layer's Xfn, for every subsequent layer — of shape (seq_len, d_model).

$$Xn1 = LayerNorm(Xin)$$

For each head h = 1, …, heads, the following is computed independently, using that head's own parameters:

$$Q^{(h)} = Xn1 \cdot W_q^{(h)}, \qquad K^{(h)} = Xn1 \cdot W_k^{(h)}, \qquad V^{(h)} = Xn1 \cdot W_v^{(h)}$$

$$att^{(h)} = \text{softmax}\left(\frac{Q^{(h)} \cdot \left(K^{(h)}\right)^T}{\sqrt{qk_d}} + M\right)$$

$$attv^{(h)} = att^{(h)} \cdot V^{(h)}$$

M is the causal mask, restricting each position to attend only to itself and earlier positions, shared identically across every head.

Each head's output is concatenated along the feature dimension into a single representation, then projected back to the model dimension:

$$conAtt = \left[attv^{(1)} \mid \cdots \mid attv^{(heads)}\right]$$

$$projectedAtt = conAtt \cdot Wp$$

Residual connection is applied to produce the final output of the block:

$$Xat = Xin + projectedAtt$$

where:

- Xin, projectedAtt, Xat: the block's input and output, each of shape (seq_len , d_model)
- $Wq^{(h)}, Wk^{(}h)$: per-head query/key weight matrices, shape (d_model, qk_d); $Wv^{(h)}$:per-head value weight matrix, shape (d_model, d_v) — one such matrix per head
- $Q^{(h)}, K^{(h)}$:per-head query/key matrices, shape (seq_len, qk_d);$V^{(h)}$: per-head value matrix, shape (seq_len, d_v)

- M: the causal mask, shape (seq_len, seq_len), shared across all heads
- $att^(h)$: head h's attention weights (post-softmax), shape (seq_len, seq_len)
- $attv^(h)$: head h's attention output, shape (seq_len, qk_d)
- conAtt: the concatenation of all heads' outputs, shape (seq_len, heads·d_v)
- Wp: the output projection matrix, mapping the concatenated heads back to the model dimension, shape (heads*d_v, d_model)

Gradient

The residual connection splits the gradient into two paths, exactly as in the FFN block:

$$dXin_1 = dXat, \qquad dprojectedAtt = dXat$$

Propagating backward through the output projection:

$$dWp = conAtt^T \cdot dprojectedAtt, \qquad dconAtt = dprojectedAtt \cdot Wp^T$$

Since conAtt was formed by concatenating each head's output, $dattv$ is split back into per-head gradients by slicing the corresponding column range for each head — the reverse of concatenation:

$$dattv^{(h)} = dconAtt[:,\ (h-1)\cdot d_v : h \cdot d_v]$$

The remaining steps are computed independently for each head:

$$datt^{(h)} = dattv^{(h)} \cdot \left(V^{(h)}\right)^T, \qquad dV^{(h)} = \left(att^{(h)}\right)^T \cdot dattv^{(h)}$$

$$dQK^{(h)} = \frac{\text{softmaxGrad}\left(att^{(h)},\ datt^{(h)}\right)}{\sqrt{qk_d}}$$

using $softmaxGrad(Y, dY)$from Section 2.3, with $att^{(h)}$as the forward-pass output and $datt^{(h)}$as the upstream gradient.

$$dQ^{(h)} = dQK^{(h)} \cdot K^{(h)}, \qquad dK^{(h)} = \left(dQK^{(h)}\right)^T \cdot Q^{(h)}$$

$$dWv^{(h)} = Xn1^T \cdot dV^{(h)}, \quad dWq^{(h)} = Xn1^T \cdot dQ^{(h)}, \quad dWk^{(h)} = Xn1^T \cdot dK^{(h)}$$

$$dXn1_2^{(h)} = dV^{(h)} \cdot \left(W_v^{(h)}\right)^T + dQ^{(h)} \cdot \left(W_q^{(h)}\right)^T + dK^{(h)} \cdot \left(W_k^{(h)}\right)^T$$

Merging across heads. Since every head reads from the same shared Xn1, their individual contributions to its gradient are summed:

$$dXn1_2 = \sum_{h=1}^{heads} d\,Xn1_2^{(h)}$$

Finally, propagating backward through Layer Normalization (Section 2.4), using the attention block's own γ, β:

$$dXin_2,\ d\gamma_{att},\ d\beta_{att} = \text{layernormGrad}\left(dXn1_2,\ \hat{Xn1}_2,\ \gamma_{att},\ \sigma^2_{att},\ \varepsilon\right)$$

Combining both residual paths:

$$dXin = dXin_1 + dXin_2$$

NumPy Implementation:

```
dXin1 = dXat
dprojectedAtt = dXat

dWp = combined_att[layer].T @ dprojectedAtt
dcombined_att = dprojectedAtt @ Wp[layer].T


# split the combined gradient back into per-head slices
dattv = np.transpose(
    dcombined_att.reshape((dcombined_att.shape[0], heads, v_d)),
    axes=(1, 0, 2)
)

dV = np.transpose(att[layer], axes=(0, 2, 1)) @ dattv
datt = dattv @ np.transpose(V[layer], axes=(0, 2, 1))

dQK = softmaxGrad(att[layer], datt)
dQK_scaled = dQK / np.sqrt(qk_d)

dQ = dQK_scaled @ K[layer]
dK = np.transpose(dQK_scaled, axes=(0, 2, 1)) @ Q[layer]

dWq = Xn1[layer].T @ dQ
dWk = Xn1[layer].T @ dK
dWv = Xn1[layer].T @ dV
```

```
dXn1 = np.sum(
    dV @ np.transpose(Wv[layer], axes=(0, 2, 1))
    + dQ @ np.transpose(Wq[layer], axes=(0, 2, 1))
    + dK @ np.transpose(Wk[layer], axes=(0, 2, 1)),
    axis=0
)

dXin2, dgamma_att, dbeta_att = layernormGrad(
    dXn1, Xhat_n1[layer], gamma_att[layer], variance_n1[layer], epsilon
)

dXin = dXin1 + dXin2
```

3.4 Initial Block

Formula

Let T be the input vector of shape (1, seq_len), where each entry is the integer token ID corresponding to one input character. Wemb is the embedding matrix, of shape (vocab_size, d_model). Each token is mapped to its corresponding row of Wemb, producing E of shape (seq_len, d_model):

$$E_{i,j} = Wemb_{T_i,\, j}$$

A fixed positional encoding is then added:

$$Xp = E + PE$$

where:

- T = the input token ID vector, shape (1, seq_len)
- Wemb = the embedding matrix, shape (vocab_size, d_model)
- E = the looked-up embedding matrix, shape (seq_len, d_model)
- PE = positional encoding, a fixed function with no learnable parameters, added element-wise to Xem

Gradient :

$dXp$is already known from Section 3.3, where it is referred to as $dXin$ for the first layer.

Since PE is a fixed function with no learnable parameters — not merely a constant value, but an operation with nothing to differentiate with respect to — it contributes no gradient of its own. Addition passes the upstream gradient through to both of its inputs unchanged, so:

$$dE = dXp$$

To find $dWemb$, note that Xem is formed by selecting individual rows of Wemb — only the rows that were actually looked up receive any gradient; every other row of Wemb receives zero. Furthermore, if a given token appears at more than one position in the sequence, every occurrence contributes to the same row of Wemb, and these contributions are summed rather than overwritten:

$$dWemb_{v,j} = \sum_{i:T_i=v} d\,E_{i,j}$$

That is, row v of $dWemb$ accumulates the gradient from every sequence position i whose token ID equals v.

NumPy implementation:

```
dE = dXp
dWemb = np.zeros_like(Wemb)
np.add.at(dWemb, T, dE)
```

Section 4: Training & Empirical Verification

4.1 Training Setup & Configuration

To evaluate the end-to-end implementation, the complete Transformer was trained as a character-level language model on the Tiny Shakespeare dataset (1,115,394 characters). The corpus was split 90 % for training (~1,003,854 characters) and 10 % for validation (~111,540 characters). The character vocabulary contains 65 unique tokens (vocab_size = 65).

The architectural dimensions and optimization hyperparameters are summarized below:

Hyperparameter	**Value**	**Description**
layers	4	Stacked Transformer blocks (Attention + FFN)
heads	4	Parallel attention heads per block
ed (d_model)	128	Token embedding and hidden dimension
qk_d	32	Query and key projection dimension per head
v_d	32	Value projection dimension per head

ffn_wd	512	Feedforward hidden layer width (4 x ed)
block_size (seq_len)	64	Context window length (seq_lenokens)
batch_size	32	Mini-batch size per Adam step
base_alpha	0.001	Peak learning rate reached after warmup
warmup_steps	500	Linear learning rate warmup schedule
decay_rate	1e-5	Inverse-time learning rate decay rate
beta1, beta2	0.9, 0.999	Adam first- and second-moment decay rates
epsilon	1e-8 (Adam), 1e-7 (LN)	Numerical stability constants
total_steps	4,000	Total gradient accumulation steps

Optimization used the Adam update rule derived in Section 2.2. The learning rate warms up linearly from 0 to 0.001 over the first 500 steps to prevent unstable updates when second-moment estimates are noisy, followed by an inverse-time decay schedule: alpha_t = base_alpha / (1 + decay_rate * (seq_len - warmup_steps)).

4.2 Loss Convergence & Training Dynamics

The model was trained for 4,000 steps. Cross-entropy loss was evaluated periodically on random mini-batches from both the training and validation sets (Figure 4.1).

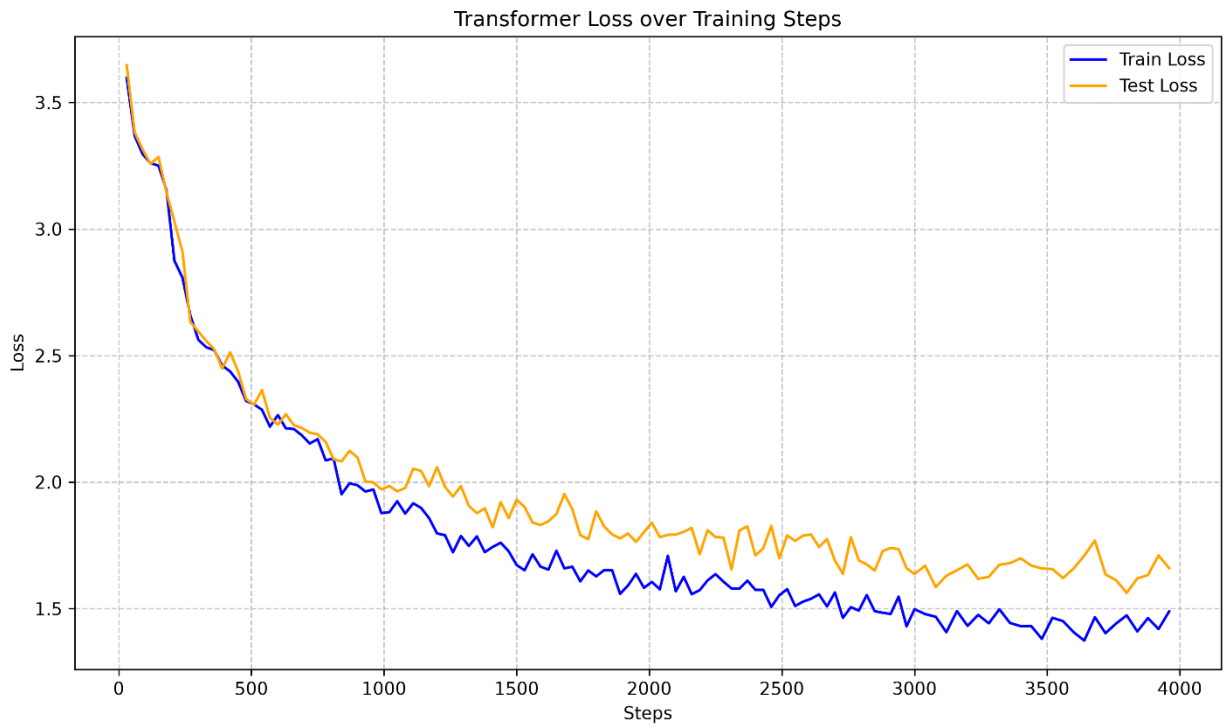


Figure 4.1: Training and validation loss curves over 10,000 steps on Tiny Shakespeare.

Key observations:

1. **Rapid initial drop (0–1,000 steps):** Loss falls from ~3.65 to ~1.60 as the model quickly learns the unigram distribution and frequent bigrams.

2. **Continued improvement (1,000–4,000 steps):** Training loss declines further to ~1.45 while validation loss stabilizes around 1.65–1.70, producing a modest generalization gap of ~0.20–0.25. This behavior is expected for a small character-level Transformer trained without dropout or weight decay on a 1 MB corpus.

4.3 Qualitative Text Generation

To evaluate generation quality, text was sampled autoregressively from the model using the procedure in sample.py. Generation began from a newline token and proceeded token-by-token with temperature = 0.8. The table below shows representative outputs at successive training stages (the full samples are provided in the accompanying files).

Step	**Loss**	**Sample Generated Output**

600	~2.0-2.1	Mostly random character sequences with occasional short English-like fragments. Character-name patterns begin to appear (e.g. “IOCTTINTEH:”, “AUCANK:”), indicating the model has started to learn the dialogue format of the dataset.
1,200	~1.7–1.8	Recognizable character names become more frequent (PENENTAR, LOAUCESTER, KING ANG ELOT, POME). Some multi-word phrases emerge, but spelling and syntax remain heavily broken.
1,800	~1.6	Longer sequences of English words appear. The model produces partial sentences and begins to capture some Elizabethan-style phrasing, though coherence is still limited.
3,000–3,200	~1.50	Noticeable gains in readability. Some lines approach coherent Shakespearean dialogue, although misspellings and incomplete thoughts remain common.
4,000	~1.45	est overall quality in the run. The model reliably produces the CHARACTER: dialogue format and generates multi-sentence passages that are partially fluent, though still far from fully grammatical or metrically consistent.

These samples illustrate a clear progression: the model first acquires the surface statistics of the corpus (character frequencies and the dialogue layout), then gradually improves local word formation and

short-range syntax. Even at 4,000 steps the generated text remains imperfect—consistent with a small character-level model (4 layers, $d_{\text{model}} = 128$) trained without regularization on a modest dataset.

Conclusion

Across this two-part series the complete design, analytical derivation, and pure-NumPy implementation of a decoder-only Transformer language model have been presented. Part 1 described the architectural structure, forward dataflow, and key design choices (pre-LN residual connections, causal masking, and independent per-head projections). Part 2 derived the exact gradients for every component—multi-head attention, the GELU feed-forward network, Layer Normalization, the unembedding layer, and the Adam optimizer—and demonstrated that the resulting implementation can be trained successfully as a character-level language model on the Tiny Shakespeare corpus.

The training run confirms that the hand-derived, vectorized backpropagation is functionally correct: the model exhibits stable optimization, a steadily decreasing training loss, and the progressive acquisition of character-level and short-range syntactic patterns. While a decreasing loss does not constitute a formal verification of every gradient, the observed learning dynamics provide strong practical evidence that the implementation is sound.

By exposing every matrix multiplication, residual connection, and gradient computation, the pure-NumPy approach offers a transparent window into the internal mechanics of Transformer models that is often obscured by high-level frameworks.